UNIVERSIDAD DE PAMPLONA
FACULTAD DE INGENIERÍAS Y ARQUITECTURA
DEPARTAMENTO E.E.S.T. — INGENIERÍA DE TELECOMUNICACIONES

MANUAL TÉCNICO DEL SISTEMA

Arquitectura, Contratos de Intercambio, Módulos por Capas (1 a 7), RAG BM25, Inferencia Local y Orquestación Agéntica

Proyecto de Grado: Benchmark Reproducible para Diagnóstico de Fallas L7 con SLMs Locales

Autor: Juan David León Arrieta (juan.leon22@unipamplona.edu.co)
Director: Mg. Richard Eliseo Mendoza Gáfaro (richard.mendoza@unipamplona.edu.co)
Versión del Software: v1.0.0 • **Fecha:** Noviembre 2026
Arquitectura: 7 Capas Desacopladas • RAG BM25 (≤0.05 MB) • LangGraph Read-Only • Typer CLI + FastAPI/HTMX
Entorno de Ejecución: 100% Local • 8 GB VRAM • Ollama / llama.cpp (GGUF Q4_K_M)

1. Introducción y Marco del Sistema

NetDiagBench-L7 es un entorno experimental reproducible concebido para evaluar rigurosamente la capacidad diagnóstica de modelos de lenguaje pequeños (SLMs) ejecutados de forma 100% local en hardware de consumo restringido (GPU con 8 GB de VRAM, 32 GB de RAM del sistema). El sistema aborda fallas complejas en la capa de aplicación sobre IP (Capa 7 OSI), concentrándose en servicios críticos como DNS (BIND9), servidores web HTTP/HTTPS (Nginx) y pasarelas de API (API Gateway).

Para garantizar reproducibilidad absoluta, rigor científico y aislamiento estricto de red, el software opera bajo seis principios inquebrantables:

- **Ejecución 100% Local:** Cero telemetría hacia la nube y prohibición de APIs comerciales externas.
- **Hardware Acotado:** Modelos cuantizados en 4 bits (GGUF Q4_K_M) con consumo de VRAM inferior a 8.0 GB.
- **Terna Homogénea de Modelos:** Qwen3-8B (8.2B), GLM-4-9B-Chat (9.0B) y Phi-4-mini-instruct (3.8B).
- **RAG Ultraliviano:** Recuperador léxico acotado BM25 con huella en memoria RAM ≤ 0.05 MB (límite arquitectónico: 0.45 GB).
- **Seguridad Read-Only:** El agente de diagnóstico opera herramientas de red exclusivamente en modo de solo lectura (dig, curl, openssl).
- **Determinismo Experimental:** Temperatura fijada en 0.0, semillas pseudoaleatorias reproducibles [42, 1337, 2026] y 3 repeticiones por escenario.

CONTROL DE REPRODUCIBILIDAD MLOPS: *El sistema implementa trazabilidad MLOps completa: cada inferencia y veredicto registra el commit hash de Git, la versión del motor Ollama, huella de hardware del host, tokens de razonamiento CoT (<think>) y digest criptográfico SHA-256.*

2. Arquitectura Global en 7 Capas Desacopladas

La arquitectura del sistema sigue un modelo estrictamente desacoplado en 7 capas funcionales. Cada capa se encuentra encapsulada en su propio paquete dentro de src/ y se comunica con las capas adyacentes a través de contratos tipados con Pydantic v2 (src/core/contracts.py).

Capa	Agente / Subsistema	Artefacto Emitido	Módulo en src/	Recurso Clave
1. Laboratorio	ScenarioLabAgent	`escenario.yaml + Logs`	`src/layer1_lab/`	Docker / Containerlab / BIND9 / Nginx
2. Preproceso	DataPipelineAgent	`ventana.jsonl`	`src/layer2_pipeline/`	Drain3 (Log Clustering) + Jinja2
3. RAG	RAGKnowledgeAgent	`contexto.md`	`src/layer3_rag/`	BM25 Retriever (RAM <= 0.05 MB)
4. Inferencia	InferenceEngineAgent	`respuesta.json`	`src/layer4_inference/`	Ollama / GGUF Q4_K_M (VRAM <= 8GB)
5. Orquestación	DiagnosticOrchestratorAgent	`traza.jsonl`	`src/layer5_orchestrator/`	LangGraph StateGraph + Tools Read-Only
6. Interfaz	InterfaceCLIAgent	`diagnostico.json/md`	`src/layer6_interface/`	Typer CLI + FastAPI / HTMX / SSE
7. Evaluación	BenchmarkEvaluatorAgent	`resultados.csv`	`src/layer7_evaluator/`	psutil / SciPy (Kruskal, Spearman, d)

3. Especificación de Contratos de Datos (Pydantic v2)

Los contratos definidos en src/core/contracts.py garantizan la interoperabilidad sin acoplamiento directo entre las capas. Entre los principales modelos de datos destacan:

- **EscenarioLabContract:** Modela el escenario inyectado, categoría (DNS, HTTP_WEB, API_GATEWAY, TLS), clase de falla (18 clases normalizadas), topología de red y ficha de Ground Truth (causa raíz, componente afectado, acción de mitigación y evidencia mínima requerida).
- **VentanaLogContract:** Agrupa los registros normalizados por Drain3 (ParsedLogEntry con template_id, parámetros extraídos y nivel de severidad) junto a la telemetría resumida (tasa de error, latencia p50/p95).
- **ContextoRAGContract:** Contiene la consulta realizada, los fragmentos normativos recuperados (RAGDocumentChunk con score BM25 y documento fuente), la concatenación Markdown formateada y la huella de memoria RAM consumida.
- **RespuestaInferenciaContract:** Captura la respuesta cruda del modelo, texto de razonamiento CoT desacoplado de bloques <think>, tokens de entrada/salida, TTFT (ms), velocidad de decodificación (tok/s) y pico de VRAM.
- **TrazaOrquestacionContract:** Bitácora de ejecución del grafo LangGraph con cada paso de herramienta ejecutado (ToolExecutionStep), comando exacto, salida obtenida e hipótesis contrastada.
- **DiagnosticoFinalContract:** Dictamen final estructurado con causa raíz, componente identificado, acción recomendada, nivel de confianza, respaldado en evidencia y penalización por herramientas bloqueadas.
- **ResultadoEvaluacionContract:** Fila canónica del dataset resultados.csv (27 columnas) integrando precisión booleana, coincidencia parcial continua (0.0-1.0), rúbrica explicativa (0-5), latencias p50/p95, energía estimada (kWh) y emisiones de CO2 (kg).

4. Catálogo de 18 Escenarios de Falla L7 sobre IP

El catálogo cubre 18 anomalías operativas de alta recurrencia en infraestructuras de telecomunicaciones e Internet, organizadas en cuatro dominios:

ID Escenario	Categoría	Clase de Falla	Componente Afectado	Causa Raíz Esperada
`DNS-DEL-01`	DNS	DELEGATION_ERROR	`bind9-zone-delegation`	Error de delegación NS cojo (Lame Delegation)
`DNS-NXD-01`	DNS	NXDOMAIN	`bind9-authoritative`	Registro de zona faltante o FQDN inexistente (RCODE 3)
`DNS-REF-01`	DNS	DNS_REFUSED	`bind9-authoritative`	Consulta rechazada por lista de control de acceso ACL (RCODE 5)
`DNS-SRV-01`	DNS	SERVFAIL	`bind9-recursive-resolver`	Falla de resolución recursiva o timeout hacia servidor raíz
`DNS-TRUNC-01`	DNS	DNS_TRUNCATED_EDNS	`bind9-resolver-firewall`	Respuesta truncada TC=1 por buffer EDNS0 bloqueado en firewall
`HTTP-POOL-01`	API_GATEWAY	CONNECTION_POOL_EXHAUSTED	`upstream-backend-api`	Agotamiento de pool de conexiones concurrentes en backend
`HTTP-403-01`	API_GATEWAY	HTTP_403_FORBIDDEN	`nginx-api-gateway`	Acceso denegado por reglas de autorización o IP restringida
`HTTP-429-01`	API_GATEWAY	HTTP_429_RATE_LIMITED	`nginx-rate-limiter`	Límite de tasa de peticiones excedido (Rate Limiting activo)
`HTTP-500-01`	HTTP_WEB	HTTP_500_INTERNAL	`backend-application-code`	Excepción no controlada en el código de la aplicación backend
`HTTP-502-01`	API_GATEWAY	HTTP_502_BAD_GATEWAY	`upstream-backend-api`	Conexión rechazada o caída del servicio upstream backend
`HTTP-503-01`	HTTP_WEB	HTTP_503_SERVICE_UNAVAILABLE	`backend-application-pool`	Servicio sobrecargado o en mantenimiento temporal
`HTTP-504-01`	API_GATEWAY	HTTP_504_GATEWAY_TIMEOUT	`upstream-backend-database`	Timeout de espera de respuesta desde el microservicio backend
`NET-LAT-01`	HTTP_WEB	NETWORK_LATENCY_TC	`network-infrastructure`	Degradación por latencia anómala inyectada con tc-netem

HTTP-ORIGIN-01	API_GATEWAY	ORIGIN_DOWN	upstream-cluster	Servidor de origen fuera de línea o sin conectividad de red
TLS-CHN-01	TLS	TLS_CHAIN_INVALID	nginx-tls-termination	Cadena de confianza rota por omisión de certificado intermedio CA
TLS-CIPH-01	TLS	TLS_CIPHER_MISMATCH	nginx-tls-crypto-engine	Negociación fallida por conjuntos de cifrado (ciphers) incompatibles
TLS-EXP-01	TLS	TLS_EXPIRED_CERT	nginx-tls-termination	Certificado digital X.509 expirado en endpoint seguro
TLS-SNI-01	TLS	TLS_SNI_MISMATCH	nginx-tls-termination	Discrepancia entre Server Name Indication (SNI) y certificado presentado

5. Base de Conocimiento Normativo RAG BM25

El módulo src/layer3_rag/retriever.py implementa un motor RAG basado en recuperación léxica acotada Okapi BM25 mediante langchain_community.retrievers.BM25Retriever. A diferencia de los enfoques vectoriales densos tradicionales que requieren modelos de embeddings de gran tamaño y bases de datos vectoriales en RAM, BM25 opera con una huella extremadamente ligera:

- **Consumo de RAM Verificado:** ≤ 0.05 MB (frente al límite máximo estipulado de 0.45 GB).
- **Corpus Normativo:** 36 fragmentos técnicos indexados derivados de estándares de la IETF: **RFC 1035** (DNS), **RFC 9110** (Semántica HTTP), **RFC 8446** (Protocolo TLS 1.3) y Procedimientos Operativos Estándar (SOP) de telecomunicaciones.
- **Parámetro k de Recuperación:** Top-k fijado en 2 fragmentos altamente relevantes incorporados dinámicamente al prompt.

6. Subsistema de Inferencia Local y Terna de Modelos

El motor LocalInferenceEngine (src/layer4_inference/engine.py) interactúa directamente con el daemon local de Ollama (o emulador sintético calibrado en ausencia de GPU activa). Todos los modelos operan con temperatura = 0.0 y cuantización Q4_K_M:

Modelo	Proveedor	Parámetros	Licencia	VRAM Aprox.	Ventana Contexto	Mecanismo Tools
Qwen3-8B	Alibaba	8.2B (6.95B sin emb)	Apache 2.0	5.5 GB	32,768 tok	Nativo / Razonamiento conmutable
GLM-4-9B-Chat	Zhipu AI	9.0B	GLM-4 License	5.9 GB	131,072 tok	Berkeley Function Calling (81%)
Phi-4-mini-instruct	Microsoft	3.8B	MIT	2.8 GB	131,072 tok	JSON Schema Tool Format

Medición de Telemetría: El subsistema instrumenta con precisión de microsegundos el *Time to First Token* (TTFT = t_primer_token - t_envio_prompt) y la velocidad de decodificación *Tokens por Segundo* (tok/s = tokens_generados / t_generacion). Asimismo, desacopla mediante expresiones regulares los bloques de razonamiento interno <think>...</think> para auditar la cadena de pensamiento (Chain-of-Thought).

7. Orquestación Agéntica LangGraph y Seguridad Read-Only

El diagnóstico de fallas no se realiza en una sola llamada generativa pasiva, sino a través de un grafo de estados determinista (src/layer5_orchestrator/agent.py) construido con **LangGraph**. El ciclo operativo transita por 6 nodos:

1. ingest_telemetry: Carga la ventana de logs normalizada con Drain3 y métricas de error.

2. retrieve_rag_knowledge: Recupera fragmentos de RFCs relevantes según el síntoma observado.
3. formulate_hypothesis: El SLM propone una hipótesis técnica y selecciona una herramienta de verificación.
4. execute_read_only_tool: Ejecuta el comando de red verificado por la lista blanca de seguridad.
5. evaluate_evidence: El agente contrasta la salida del comando con la hipótesis planteada.
6. synthesize_diagnosis: Emite el veredicto estructurado con causa raíz, componente y mitigación NOC.

SEGURIDAD OPERATIVA Y SANDBOXING READ-ONLY: Política de Solo Lectura: *src/layer5_orchestrator/tools.py restringe la ejecución a una lista blanca estricta de utilitarios (dig, curl, openssl s_client). Cualquier intento de concatenación de comandos (;, &&, |), redirección o modificación del sistema operativo (rm, iptables, systemctl) es interceptado y bloqueado, aplicando una penalización de 1.0 punto al score del modelo.*

8. Métricas de Evaluación y Análisis Estadístico SciPy

El módulo src/layer7_evaluator/ somete el desempeño de los modelos a un análisis multidimensional riguroso:

- **Exactitud de Causa Raíz (Booleana):** Concordancia estricta con la ficha de Ground Truth mediante conceptos canónicos normalizados (configs/evaluator_concepts.yaml).
- **Score de Coincidencia Parcial (0.0 - 1.0):** Evaluación léxico-semántica continua calculada mediante distancia de tokens y cobertura de conceptos clave.
- **Rúbrica de Explicación (0 a 5 puntos):** Califica coherencia técnica, fundamentación con normas RFC, correcta identificación del componente y pertinencia de la mitigación sugerida.
- **Score de Eficiencia Compuesto (OE4):** Combina precisión diagnóstica (40%), rúbrica de explicación (20%), velocidad de inferencia normalizada (20%) y eficiencia de VRAM (20%).

- **Pruebas de Hipótesis SciPy (Tarea 4.1.2):**
  - *Kruskal-Wallis H-test:* Prueba no paramétrica para contrastar diferencias entre los 3 modelos (confirmando significancia $p < 0.001$ en TTFT, tok/s, latencia y energía).
  - *Correlación de Spearman (ρ) y Pearson (r):* Evalúa la relación entre tamaño de parámetros y latencia/precisión.
  - *Tamaño del Efecto d de Cohen:* Cuantifica la magnitud de la diferencia entre pares (confirmando efectos grandes $|d| \geq 0.8$ en 6 de 9 comparaciones).
  - *Intervalos de Confianza al 95%:* Calculados vía estimación asintótica / bootstrap para cada métrica clave.

9. Referencia de la API REST y Comandos CLI

El sistema provee interfaces duales: una interfaz de línea de comandos basada en **Typer** (benchapp) para automatización por lotes, y un servidor web **FastAPI** con endpoints REST y streaming por Server-Sent Events (SSE).

Método	Ruta del Endpoint	Módulo	Descripción Funcional
GET	`/api/scenarios`	`web.py`	Lista los 18 escenarios de falla con su Ground Truth
GET	`/api/models`	`web.py`	Consulta la terna de modelos y estado en Ollama
POST	`/api/models/pull`	`web.py`	Descarga dinámica de nuevos modelos SLM desde Ollama
GET	`/api/health`	`web.py`	Diagnóstico de salud del subsistema Ollama, VRAM y modos
POST	`/api/ejecutar`	`web.py`	Inicia diagnóstico interactivo vía HTMX devolviendo visor SSE
GET	`/stream-diagnostico`	`observability.py`	Flujo SSE en tiempo real de eventos del grafo LangGraph
POST	`/api/execute-suite`	`observability.py`	Ejecuta la suite continua de 18 escenarios en secuencia
GET	`/vistas/comparativo`	`web.py`	Fragmento HTML de la matriz comparativa de modelos (OE4)

GET	`/vistas/correlacion`	`web.py`	Fragmento HTML del análisis correlacional y estadístico SciPy
GET	`/api/export/excel`	`web.py`	Exporta libro Excel multioja profesional (.xlsx)
GET	`/api/export/pdf/comparative`	`web.py`	Descarga reporte institucional en PDF del OE4

Comandos de la CLI (benchapp):

```
# Listar catálogo de escenarios y modelos
.venv\Scripts\benchapp.exe list-scenarios
.venv\Scripts\benchapp.exe list-models

# Consulta léxica RAG contra corpus normativo
.venv\Scripts\benchapp.exe rag-query "falla de certificado expirado"

# Ejecutar diagnóstico de un escenario individual
.venv\Scripts\benchapp.exe run-scenario DNS-NXD-01 --model phi-4-mini-instruct --seed 42

# Ejecutar benchmark automatizado (3 modelos x 18 escenarios x 3 semillas)
.venv\Scripts\benchapp.exe run-benchmark --seeds 42,1337,2026 --repetitions 1

# Análisis estadístico y exportación de entregables
.venv\Scripts\benchapp.exe analyze-correlation
.venv\Scripts\benchapp.exe export-excel
.venv\Scripts\benchapp.exe export-pdf --type all

# Iniciar servidor web de observabilidad
.venv\Scripts\benchapp.exe serve-dashboard --host 127.0.0.1 --port 8000
```

10. Procedimiento de Despliegue y Batería de Pruebas

El framework cuenta con una suite integral de **160 pruebas automatizadas** (158 unitarias deterministas y 2 de integración en contenedores Docker reales):

- tests/test_contracts.py: Validación de esquemas Pydantic y serialización JSON de las 7 capas.
- tests/test_layer1_lab.py: Carga e integridad técnica de los 18 archivos YAML del catálogo.
- tests/test_layer2_pipeline.py: Agrupamiento de logs con Drain3 y renderizado de plantillas Jinja2.
- tests/test_layer3_rag.py: Medición de huella RAM de BM25 (≤ 0.05 MB) y relevancia de fragmentos.
- tests/test_layer4_inference.py: Telemetría de TTFT, tok/s y desacoplamiento de bloques <think>.
- tests/test_layer5_orchestrator.py: Transiciones del grafo LangGraph y lista blanca de herramientas de red.
- tests/test_statistical_analysis.py: Verificación de cálculos de Kruskal-Wallis, Spearman y Cohen's d.
- tests/test_exporters.py: Generación correcta de archivos Excel (.xlsx) y reportes en PDF.

Ejecución de Pruebas:

```
# Ejecutar todas las pruebas unitarias deterministas (tiempo < 5 segundos)
.venv\Scripts\pytest.exe -v tests/

# Ejecutar verificación integral de pre-vuelo del sistema
.venv\Scripts\benchapp.exe doctor
```